

Exp 2 : To create basic Blockchain with sample transactions

Name : Abhishek Pandey

SAP : 60019220110

Roll No : B073

```
import hashlib
import time

class Block:
    def __init__(self, tx, prev_hash):
        self.ts = time.time()
        self.tx = tx
        self.prev_hash = prev_hash
        self.hash = self.calc_hash()

    def calc_hash(self):
        s = str(self.ts) + str(self.tx) + str(self.prev_hash)
        return hashlib.sha256(s.encode()).hexdigest()

class Blockchain:
    def __init__(self):
        self.chain = []
        self.genesis()

    def genesis(self):
        self.chain.append(Block("Genesis Block", "0"))

    def add(self, tx):
        prev_hash = self.chain[-1].hash
        self.chain.append(Block(tx, prev_hash))

    def show(self):
        for b in self.chain:
            print(f"Time : {b.ts}")
            print(f"Tx : {b.tx}")
            print(f"Prev : {b.prev_hash}")
            print(f"Hash : {b.hash}")
            print(".....Welcome we are creating blockchain.....")

bc = Blockchain()
bc.add("HELLO")
bc.add("BYE")
bc.add("Pratham")
bc.add("Understood how creation of blockchain work")
bc.show()
```

Output :

```
Time : 1756652464.801658
Tx : Genesis Block
Prev : 0
Hash : b1f8679fa8440d97260092c016bef87e995870e5f861b3caf05d29d0182ce6ad
.....Welcome we are creating blockchain.....
Time : 1756652464.8017728
Tx : HELLO
Prev : b1f8679fa8440d97260092c016bef87e995870e5f861b3caf05d29d0182ce6ad
Hash : 3c48ee6da2c85492804eae2aabb1124de4bb428b9c2108c7fa35080b92d55d82
.....Welcome we are creating blockchain.....
Time : 1756652464.801823
Tx : BYE
Prev : 3c48ee6da2c85492804eae2aabb1124de4bb428b9c2108c7fa35080b92d55d82
Hash : 0f73d1ec937d1c7246256ee3d2fc8c367b2be6dcacd324717adfef02ac5d0e17
.....Welcome we are creating blockchain.....
Time : 1756652464.8018575
Tx : Python
Prev : 0f73d1ec937d1c7246256ee3d2fc8c367b2be6dcacd324717adfef02ac5d0e17
Hash : 0d5f48515cfb1f9435da2a896ec321de796c9cb864c114e2a87bc6da8e521c81
.....Welcome we are creating blockchain.....
Time : 1756652464.8018918
Tx : Understood how creation of blockchain work
Prev : 0d5f48515cfb1f9435da2a896ec321de796c9cb864c114e2a87bc6da8e521c81
Hash : 69def6e985b431e73a2668245f44751179685d04e8d4fd276fbe9d60f0940b24
.....Welcome we are creating blockchain.....
```

```

import hashlib
import time

# --- Merkle Tree Helper Functions ---

def hash_data(data):
    return hashlib.sha256(data.encode()).hexdigest()

def merkle_root(transactions):
    if not transactions:
        return None

    hashes = [hash_data(tx) for tx in transactions]

    while len(hashes) > 1:
        # If odd number of hashes, duplicate the last one
        if len(hashes) % 2 == 1:
            hashes.append(hashes[-1])

        new_hashes = []
        for i in range(0, len(hashes), 2):
            combined = hashes[i] + hashes[i+1]
            new_hashes.append(hash_data(combined))
        hashes = new_hashes

    return hashes[0]

def get_merkle_proof(transactions, target_tx):
    """
    Returns the Merkle proof path (sibling hashes) for the given transaction.
    """
    if target_tx not in transactions:
        return []

    def hash_pair(a, b):
        return hash_data(a + b)

    idx = transactions.index(target_tx)
    hashes = [hash_data(tx) for tx in transactions]
    proof = []

    while len(hashes) > 1:
        if len(hashes) % 2 == 1:
            hashes.append(hashes[-1])

        sibling_idx = idx ^ 1 # index of sibling node
        proof.append(hashes[sibling_idx])

        idx = idx // 2
        new_hashes = []
        for i in range(0, len(hashes), 2):
            new_hashes.append(hash_pair(hashes[i], hashes[i+1]))
        hashes = new_hashes

    return proof

```